

Excavator3000

Project Features and DQN Training Guide

Document status: Phase 2 implementation

Simulation: Webots

Training framework: PyTorch DQN

Deployment target: ESP32 RC car with TT motors and 68 mm wheels

1. Project purpose

Excavator3000 is a reinforcement-learning controller developed in Webots and intended for later deployment to an ESP32-based RC car. The learned model makes high-level driving decisions. The hardware layer remains responsible for converting those decisions into tuned TT motor PWM, RPM, acceleration, and steering behavior.

The simulator and real vehicle share a normalized model interface. This reduces dependence on Webots-specific sensor units and motor speeds.

2. Current phase status

Phase	Status	Result
Phase 1 - Environment foundation	Complete	Webots control, reset, sensors, observations, goal, collision, rewards, replay memory, and HUD
Phase 2 - DQN training pipeline	Complete in code	Network, agent, replay learning, target network, checkpoints, evaluation, CSV metrics, and tests
Phase 3 - Model conversion	Not started	PyTorch export followed by TensorFlow Lite conversion and numerical validation
Phase 4 - ESP32 integration	Not started	Quantized inference, real sensor calibration, and hardware motor tuning

The Phase 2 pipeline is ready for a real Webots training run. Training quality still needs to be measured over long runs and reward settings may need tuning from observed behavior.

3. System architecture

```

Webots world
|
v
RobotController
|-- 8 raw distance readings -----> collision detector and HUD
|-- lookup-table distances -----> normalized proximity values
|-- linear/angular velocity -----> normalized motion values
|
v
10-value observation
|
v
DQN online network ---> 7 Q-values ---> epsilon-greedy action
|                                     |
|                                     v
|                                     wheel motor command
|
+<--- replay batch <--- uniform replay buffer
|
+-- Bellman target from target network
+-- Huber loss, Adam update, gradient clipping

```

The observer receives UDP telemetry over localhost. It can be disabled during training so rendering does not slow simulation.

4. Webots environment

4.1 Robot and track

- Configured vehicle width: 0.145 m
- Configured vehicle length: 0.255 m
- Wheel radius used by simulation telemetry: 0.0205 m
- Real target wheel diameter: 68 mm
- Track arena configuration: 10 m by 10 m
- Fixed mapped start position and heading
- Goal node requirement: `DEF GOAL Solid`
- Goal detection region: 1.7 m by 0.1 m with 0.03 m tolerance
- Maximum episode length: 500 environment steps

The active Webots world is stored in the surrounding Webots project at `worlds/car_env_test_v2.wbt`. The Git repository currently begins at the `controllers/rl_controller` directory, so the world itself is not part of this controller repository.

4.2 Episode lifecycle

Each episode performs the following sequence:

1. Stop both motors.
2. Restore the configured start position and heading.
3. Reset Webots physics to remove momentum.
4. Allow sensors and physics to settle.
5. Read the initial 10-value observation.
6. Execute agent decisions until goal, collision, timeout, or simulation stop.
7. Record an explicit termination reason and reset for the next episode.

Collision has priority over goal detection. A vehicle touching a wall cannot be counted as successful merely because its center is inside the goal region.

4.3 Termination reasons

Reason	Terminal	Meaning
running	No	Episode continues
goal_reached	Yes	Vehicle center entered the configured goal region
collision	Yes	At least one raw sensor exceeded 3900
timeout	Yes	Episode reached 500 steps
simulation_stopped	Yes	Webots returned its stop signal

5. Agent interface

5.1 Observation space

The model receives exactly 10 float values in this order:

Index	Input	Range	Interpretation
0	Front proximity	0 to 1	0 clear, 1 extremely close
1	Back proximity	0 to 1	0 clear, 1 extremely close
2	Left proximity	0 to 1	0 clear, 1 extremely close
3	Right proximity	0 to 1	0 clear, 1 extremely close
4	Left-front proximity	0 to 1	0 clear, 1 extremely close
5	Right-front proximity	0 to 1	0 clear, 1 extremely close
6	Left-back proximity	0 to 1	0 clear, 1 extremely close
7	Right-back proximity	0 to 1	0 clear, 1 extremely close
8	Signed forward speed	-1 to 1	Reverse to maximum forward speed
9	Signed turn rate	-1 to 1	Right turn to left turn

Webots lookup-table distances are normalized against the 0.5 m sensor range. Forward speed is projected onto the robot's local forward axis instead of using unsigned world speed. All model inputs are clipped to their documented ranges.

Raw sensor readings remain separate. Collision detection never depends on normalized model inputs.

5.2 Action space

Action	Decision	Left/right drive ratios
0	Forward	+1.00, +1.00
1	Turn left	+0.45, +1.00
2	Turn right	+1.00, +0.45
3	Reverse	-0.80, -0.80
4	Stop	0.00, 0.00
5	Reverse left	-0.40, -0.80
6	Reverse right	-0.80, -0.40

These are high-level decisions. The real ESP32 motor layer can map them to calibrated PWM targets without retraining the decision network, provided the real observations use matching normalization.

6. Reward system

The reward module avoids paying the agent merely to survive. Waiting, stopping, circling until timeout, and approaching a wall at speed are all unfavorable.

6.1 Terminal rewards

Result	Reward
Goal reached	+100 to +150
Collision	-100
Timeout	-50
Simulation stopped	0

Goal reward decreases linearly with elapsed episode time:

```
goal_reward = 100 + 50 * (1 - step / maximum_steps)
```

Fast completion therefore earns more, while late completion is still strongly better than failure.

6.2 Running reward

```
danger = max(front, left-front, right-front)
positive_speed = max(0, normalized_forward_speed)

safe_motion = 0.03 * positive_speed * (1 - danger)
danger_penalty = -0.20 * danger^2 * (1 + positive_speed)
time_penalty = -(0.005 + 0.020 * episode_progress)
stuck_penalty = -0.020 when absolute_forward_speed is below 0.05
```

The maximum safe-motion shaping reward is deliberately small. A vehicle that drives safely for the full episode and then times out still receives a negative total return.

The HUD telemetry exposes every reward component: motion, danger, time, stuck, and terminal.

7. DQN implementation

7.1 Network

```
10 inputs
-> Dense 64 + ReLU
-> Dense 64 + ReLU
-> Dense 7 Q-values
```

- Trainable parameters: 5,319
- Approximate float32 weight size: 20.8 KiB

- Output: one Q-value for each discrete action
- Deployment-friendly operations: fully connected layers and ReLU

The training dependency stays on the PC. Only converted model weights and a small inference runtime are intended for the ESP32.

7.2 Learning algorithm

- Algorithm: standard Deep Q-Network
- Discount factor: 0.99
- Optimizer: Adam
- Learning rate: 0.001
- Loss: Huber loss (`SmoothL1Loss`)
- Gradient norm clipping: 10.0
- Target network synchronization: every 1,000 optimizer updates
- Training frequency: every environment step after replay warmup
- Device selection: CUDA when available, otherwise CPU

Bellman target:

```
target = reward + gamma * (1 - done) * max(target_network(next_state))
```

7.3 Exploration

Epsilon-greedy exploration starts fully random and decreases linearly:

- Start epsilon: 1.00
- Final epsilon: 0.05
- Decay duration: 50,000 environment steps
- Evaluation epsilon: 0.00

7.4 Replay memory

- Type: uniform replay
- Capacity: 50,000 transitions
- Batch size: 64
- Learning warmup: 2,000 transitions
- Sampling: uniform without removal

Each transition stores:

```
(state, action, reward, next_state, done, termination_reason)
```

Replay memory is used only during PC training and is not exported to the ESP32.

7.5 Checkpoints and metrics

Training writes:

- `models/dqn_latest.pt` - periodic and final resumable checkpoint
- `models/dqn_best.pt` - highest episode-return checkpoint
- `runs/training.csv` - episode-level metrics

Checkpoints contain both networks, optimizer state, environment/training counters, episode number, architecture metadata, and episode summary. The `models` and `runs` directories are intentionally ignored by Git because they are generated experiment artifacts.

8. Observer HUD

The external Tkinter HUD receives JSON telemetry over UDP at `127.0.0.1:8765`.

Displayed information includes:

- Eight sensor rectangles around the robot
- Raw sensor values and estimated distances
- Proximity severity colors
- Actual speedometer and angular speed
- Agent action and wheel targets
- Episode, step, and termination status
- Step reward and accumulated reward
- Reward-component breakdown
- Training mode, epsilon, replay-buffer size, and latest loss

The HUD remains enabled in test and evaluation modes. It is disabled during training by default through `observer.enabled_in_training = False`. Set that option to `True` when a visible diagnostic training run is more important than simulation speed.

9. Running the project

9.1 Install training dependencies

From this controller directory:

```
python -m pip install -r requirements-training.txt
```

The checked configuration uses PyTorch 2.13 with CUDA 13 for the RTX 5060 Ti. The code also supports CPU execution by changing `training.device` to `cpu`.

Verify the installation:

```
python -c "import torch; print(torch.__version__);  
print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0))"
```

9.2 Test mode

In `config.py`:

```
"program": {  
    "mode": "test",  
    "terminal_output": True,  
    "test_action": 0  
}
```

Run the Webots world. The controller repeatedly executes the selected fixed action and displays the full HUD.

9.3 Training mode

Set:

```
"mode": "train"
```

Then run the Webots world. Training performs 1,000 episodes by default. Edit `training.episodes` for a shorter smoke test or a longer experiment.

To resume `models/dqn_latest.pt`, set:

```
"resume": True
```

9.4 Evaluation mode

After a checkpoint exists, set:

```
"mode": "evaluate"
```

Evaluation loads `models/dqn_best.pt`, disables exploration and learning, runs 20 episodes, and reports success count and average reward.

10. Configuration map

All operating values are centralized in `config.py`:

- `program` - controller mode and test action
- `simulation` - Webots time step and motor-speed ceiling
- `robot` - physical dimensions and action ratios
- `observation` - normalization dimensions and limits
- `environment` - episode, goal, collision, reset, and reward settings
- `observer` - HUD launch and telemetry settings
- `logging` - CSV experiment output
- `training` - DQN, epsilon, replay, checkpoint, and optimizer settings
- `evaluation` - evaluation count and checkpoint path

11. Source modules

File	Responsibility
<code>rl_controller.py</code>	Webots entry point and mode dispatch
<code>robot_controller.py</code>	Supervisor, motors, sensors, reset, and telemetry
<code>environment.py</code>	RL reset/step/action/observation interface
<code>episode_manager.py</code>	Goal, collision, timeout, and explicit termination
<code>observation.py</code>	Hardware-friendly normalized observation construction
<code>reward_calculator.py</code>	Time-aware safety reward and breakdown
<code>replay_buffer.py</code>	Uniform fixed-capacity experience replay
<code>dqn_network.py</code>	Compact PyTorch Q-network
<code>dqn_agent.py</code>	Exploration, learning, target network, and checkpoints
<code>dqn_trainer.py</code>	Training and evaluation episode orchestration
<code>training_logger.py</code>	CSV episode statistics
<code>hud_gui.py</code>	External real-time observer window
<code>config.py</code>	Central configuration

12. Validation completed

The automated suite currently contains 32 tests covering:

- Goal-region measurement and success detection
- Collision priority and timeout boundary
- Observation normalization, shape, bounds, heading, and reverse speed
- Replay warmup, capacity, sampling, copies, and factory settings

- Reward decay, danger, stuck behavior, terminal values, and anti-survival exploit
- DQN tensor shape, epsilon schedule, greedy action, optimizer update, target sync, and checkpoint restore
- Trainer episodes, logging, checkpoint creation, HUD integration, and evaluation isolation

Validation commands:

```
python -m unittest discover -s tests -v
python -m py_compile config.py dqn_agent.py dqn_network.py dqn_trainer.py
environment.py episode_manager.py hud_gui.py observation.py
replay_buffer.py reward_calculator.py rl_controller.py robot_controller.py
training_logger.py
```

The CUDA smoke test confirms that PyTorch places the model on the NVIDIA GeForce RTX 5060 Ti.

13. Sim-to-real boundary

The learned DQN chooses one of seven actions. It does not directly generate TT motor PWM.

The hardware team should implement:

- Real sensor calibration into the same eight 0-to-1 proximity values
- Measured forward-speed normalization
- Measured turn-rate normalization
- Action-to-wheel PWM or RPM mapping
- Motor dead-zone compensation
- Acceleration limiting
- Left/right motor balancing
- Emergency stop and independent safety checks

The rated 200-250 RPM TT motors with 68 mm wheels have a much higher theoretical free speed than the current simulated vehicle. Matching normalized observations and action meaning is more important than forcing identical raw motor commands.

14. Remaining work

Phase 2 provides the pipeline, but the following work remains before deployment:

1. Run a short Webots smoke-training session and inspect HUD/CSV metrics.
2. Run long training with the HUD disabled.
3. Measure success rate, collision rate, timeout rate, and return trend.
4. Tune rewards, epsilon decay, and checkpoint selection if behavior is unstable.
5. Add randomized safe start zones or multiple tracks for generalization.
6. Export the trained PyTorch network through a supported conversion path.
7. Compare PyTorch and converted-model outputs numerically.
8. Quantize for TensorFlow Lite Micro and validate memory use on the selected ESP32.

9. Calibrate real sensors and tune the motor-control layer.

15. Current milestone

The Webots environment foundation and complete DQN training pipeline are implemented, tested, GPU-enabled, and ready for the first controlled training run.

VERIFIED READINESS

- Phase 1 environment foundation is complete.
- Phase 2 DQN training code is complete.
- All 32 automated tests pass.
- The compact 5,319-parameter network runs on the RTX 5060 Ti through CUDA.
- Test, training, evaluation, checkpoint, logging, and observer paths are connected.

NEXT CONTROLLED ACTION

Run a short 10-20 episode Webots smoke-training session. Confirm replay warmup, loss updates, checkpoints, CSV metrics, resets, and termination reasons before beginning a long unattended training run.

STATUS: READY FOR WEBOTS SMOKE TRAINING